AKKA

Scale AI, Efficiently
The Agentic Systems Platform

Akka vs. Orkes (Conductor)

A comparison for teams building agentic AI

October 2026

Orkes Conductor orchestrates your agents; Akka runs your whole agentic system — and guarantees it. Orkes Conductor is a durable workflow and agent-orchestration engine from the Netflix Conductor lineage — proven, event-driven, and one layer. It runs agents as durable workflows, with guardrails and human approvals as steps. You bring the long-term memory, the real-time streaming, and the application platform around it. The Akka Agentic AI Platform delivers them as one system that guarantees resilience and scalability.

99.99%	99.9999%	4ms	5B
ORKES SLA (UP TO)	AKKA SLA	AKKA STATE READS	TOKENS / SEC BENCHMARK

DIMENSION	ORKES (CONDUCTOR)	AKKA
What it is	A durable workflow / agent-orchestration engine (managed Netflix Conductor)	The Akka Agentic AI Platform — guarantees resilience and scalability
Scope	Orchestration core with agents, guardrails, and human approvals; long-term memory and streaming are sourced, integrated, and operated by the customer	Orchestration, agents, memory, streaming, APIs, observability, and governance on one runtime
Availability SLA	Up to 99.99% advertised for Enterprise; published SLA terms list 99.9% and 99.0% plans, service credits as sole remedy	99.9999% — entire platform, backed by indemnities
RTO / RPO	Not published as a numeric guarantee	Sub-1-minute RTO; zero-byte RPO; active-active HA/DR
AI agents	Agents compiled into durable workflows by the Conductor agent runtime (now including Agentspan); AGENT task; MCP/API Gateway; multi-LLM orchestration	Native agents with tools, handoffs, guardrails, interaction logging
Memory	Agent session state persisted in workflow variables; long-term memory via external vector databases (Pinecone, pgvector, MongoDB Atlas)	Durable in-memory, 4ms reads / sub-10ms writes
State / durability	External datastores: PostgreSQL recommended for production, Redis + Elasticsearch/OpenSearch for throughput, MySQL and Cassandra supported; customer-operated when self-hosted	Durable sharded in-memory state, replayable from its event journal
Real-time streaming	Change-data-capture events to external brokers (Kafka, SQS, Pub/Sub); worker-polling task queues	Built in, backpressured, petabyte-scale, no external broker
Governance / EU AI Act	RBAC, configuration audit logs, secrets; inline agent and PII guardrails with human approval, on Conductor workloads only; no regulation mapping or pre-deployment classification	Aspect-woven runtime enforcement + full pre-production governance
Cost model	Custom Enterprise pricing (unpublished); free Developer Edition has no SLA; customer provisions the surrounding memory and streaming stack	Shared compute; up to 90% lower infrastructure for the same workload

Certifications	SOC 2 Type II	19 standards (SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF)

Orchestration Is One Layer; Akka Is the Full Stack

Orkes Conductor solves durable orchestration of long-running workflows and agents, and it solves it well — that is its heritage from Netflix Conductor. But an agentic AI system needs far more than orchestration, and Orkes leaves the rest to you. In Conductor, an AI agent is a workflow that calls an LLM to decide its next step, and guardrails and human approvals run as steps in that workflow. The long-term memory and the streaming tier are integrated and operated around the orchestration core, and you own every failure between them.

Everything an agentic system needs runs in one Akka runtime, so a system that compiles is ready to run in production. The Akka SDK models the entire agentic system in one place — agents, tools, durable memory, streaming, APIs, and governance — and the platform generates it, tests it, and runs it as one certified service.

Capability	Orkes (Conductor)	Akka
Workflow / agent orchestration	Yes — durable, proven	Yes
Native AI agents (tools, handoffs, streaming)	Agents are workflows; Conductor agent runtime (now including Agentspan) + MCP Gateway	Built in
Durable memory	Session state in the workflow; long-term memory via external vector DB	Built in, 4ms / sub-10ms
Real-time streaming	CDC to external brokers; no streaming engine	Built in, backpressured, petabyte-scale
HTTP / gRPC API layer	Gateway exposes workflows as endpoints	Built in
Governance / policy enforcement	Inline agent and PII guardrails, human approval; Conductor workloads only	Inline, runtime-embedded
Pre-production governance	Agent Evals before promotion; no classification or sign-offs	Classification, sign-offs, sealed posture

Availability and Disaster Recovery

Orkes Cloud advertises an availability SLA of up to 99.99% for Enterprise. Its published SLA terms list a 99.9% plan and a 99.0% plan, with service credits as the sole remedy. That is real. Two gaps matter for mission-critical agentic systems: the SLA covers the orchestration platform, and Orkes does not publish a numeric RTO/RPO guarantee. Self-hosted Conductor keeps state in external datastores that the customer provisions and operates, each under its own availability terms. The Conductor docs recommend PostgreSQL for production.

Metric	Orkes (Conductor)	Akka
Availability SLA	Up to 99.99% advertised; published terms 99.9% / 99.0%	99.9999%
Allowed downtime / year	~52 minutes (99.99%)	~31 seconds
RTO	Not published as a numeric guarantee	Sub-1 minute
RPO	Not published as a numeric guarantee	Zero byte
SLA scope	The orchestration platform	The entire platform

Orkes's SLA covers the orchestration platform. The memory and streaming services a customer integrates operate under their own SLAs — or none.

Up to 90% Cheaper to Operate

AI systems built with Akka are up to **90% cheaper to operate** than Python-based systems — a function of the infrastructure required for the same agentic transaction volume, not list price. With Orkes you run the orchestration engine plus the vector store and message brokers around it, and you provision and operate each. Self-hosted Conductor adds the datastores to that list.

Akka runs all of it on one shared-compute runtime. The efficiency comes from actor concurrency (~10T tokens/core/year vs ~2T; ~80% less compute than Python frameworks; Manulife: up to 300% more concurrency, 30–50% faster processing after porting from Python), shared compute, and micro-checkpointing. The spend is predictable — a fixed annual fee, not consumption that moves with load.

The Memory and State Gap

Self-hosted Conductor keeps execution state in **external datastores** that the customer provisions, scales, and operates. The Conductor docs recommend PostgreSQL for production, with Redis and Elasticsearch or OpenSearch for higher throughput. Agent session state persists in workflow variables. Long-term agent memory means wiring an external vector database such as Pinecone, pgvector, or MongoDB Atlas.

Akka provides durable sharded in-memory state as a platform primitive: **4ms reads / sub-10ms writes**, petabyte-scale, replayable from its event journal, with no external store to run. Agent memory is built in, not bolted on.

Continuous AI Intelligence

Akka Optimize keeps improving the AI after it ships. It runs an evaluation loop on your own production traffic, grading live interactions and training smaller specialized models on your proprietary data, so the system keeps getting better as it runs. The loop delivers three results that compound over time:

- **Model leverage** — route each request to the best model from any vendor under your policies, and reserve frontier models for the requests that need them.
- **Data sovereignty** — the specialized models are trained on your proprietary data, owned by you, and run inside your own environment.
- **Cost governance** — full visibility and control of AI spend, with savings that compound as the loop keeps running.

The evaluations run continuously inside Akka and grade traffic from agents wherever they run, in Akka or in third-party harnesses.

Governance and the EU AI Act

Orkes provides SOC 2 Type II, role-based access control, secret management, and configuration audit logs. Agent guardrails run inline on input, output, and tool arguments and can block a step, retry it, or pause it for human approval. Conductor records each guardrail verdict in the execution history and does not describe that history as tamper-evident. Orkes guardrails act only on Conductor workloads, and Orkes publishes no regulation mapping or pre-deployment classification.

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover

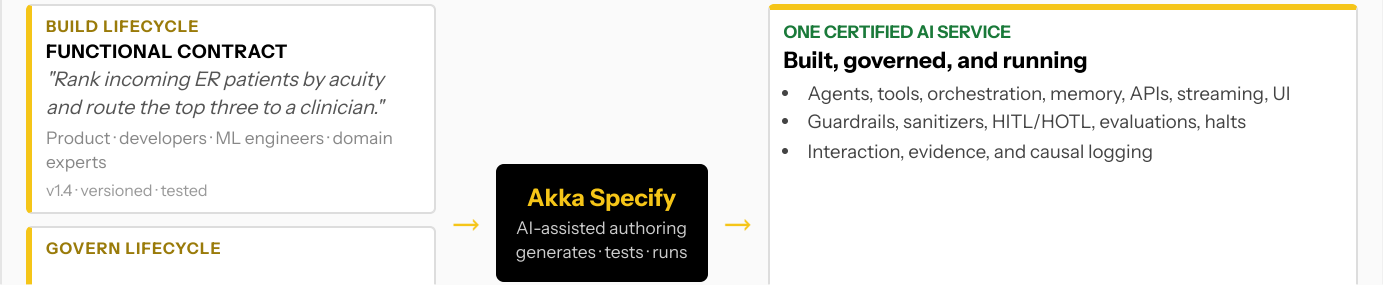
High-risk AI carries a 10-year logging-retention obligation (Art. 72).

How Akka governs

At the runtime: inline guardrails, policies, LLMs-as-a-judge, and sanitizers; hash-chained immutable evidence; HITL/HOTL control; classification against 241 regulations and frameworks and 1,718 controls before a system ships; multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify proving conformance from the running system. Akka Verify applies any control to any AI system and guarantees its enforcement.

Two Lifecycles, One Certified System

Building on Orkes means engineers defining and operating durable workflows; there is no built-in governance lifecycle and no path for a product manager, domain expert, or risk officer to contribute a governed contract. Akka runs two independent lifecycles on one platform via **Akka Specify**:



SAFEGUARD CONTRACT
"Block prohibited practices under EU AI Act Article 5; notify regulators within 24h of any incident."
Risk · security · compliance
v2.1 · versioned & tested independent of the build

Akka Verify ↻ validates the running system against both specs and fine-tunes the AI from production data.

The build lifecycle and the governance lifecycle are versioned and tested independently, by different audiences — an audience and a workflow Orkes has no equivalent for.

Real-Time Streaming at Petabyte Scale

Orkes Conductor streams workflow state changes to external brokers via change data capture (Kafka, SQS, Pub/Sub, NATS) and runs on worker-polling task queues; it has no built-in streaming engine, so real-time pipelines and backpressure are provisioned on infrastructure you operate. Akka's streaming is built into the runtime — continuous, backpressured, **petabyte-scale, in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second).

For the Buyer: Maturity, Risk, and Accountability

Orkes carries a genuine maturity advantage in orchestration: Conductor was open-sourced at Netflix in 2016, has 32.4k GitHub stars on the conductor-oss repository, and runs in production at large enterprises — Orkes maintains the project after Netflix discontinued support in December 2023. Where the lines fall:

Buyer concern	Orkes (Conductor)	Akka
Orchestration maturity	Strong — Netflix lineage, proven OSS, large deployments	Durable execution substrate; since 2009, 100,000+ deployments
Certifications & audits	SOC 2 Type II	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Scope of accountability	The orchestration platform; you integrate and operate the rest	One platform, one SLA, 24/7 SRE — Akka owns the running system
Risk transfer	Service credits on confirmed SLA breach	Availability and data-integrity guarantees backed by contractual indemnities
Funding model	Venture-funded: $60M Series B, April 2026 (AXA Venture Partners); prior $20M Series A (2024); scaling on investor capital	Profitable and self-funding; Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Budget predictability	Custom Enterprise pricing plus the surrounding stack you operate	Fixed annual fee finance can forecast

Orkes's orchestration heritage is real and durable — but Akka funds its roadmap and support from profit, not the next round, so the platform you standardize on does not depend on a venture timeline. The decision is scope and accountability: Orkes gives you a proven, event-driven orchestration engine to build a platform around; Akka gives you the platform.

Customers Running Agentic and Real-Time Systems on Akka

Manulife
2,000
developers across 100 projects on one governed platform

Tubi
5B tok/s
real-time hyper-personalization engine

Swiggy
71ms
order-assignment AI, ~50% latency reduction

John Deere
1,000+
tractor sensors turned into real-time insight

Verizon
750%
order-processing capacity gain; 6s → 2.4s response

Common Questions

We already run Orkes Conductor in production. Why add Akka?
Orkes is strong for durable orchestration — that is its Netflix Conductor heritage. If you are moving into agentic AI you also need long-term memory, real-time streaming, and governance for AI

Can we add governance on top of Orkes?
Orkes includes inline guardrails and human approval steps for agents and workflows that run on Conductor. Those guardrails cover no AI system outside Conductor and map to no named regulation. Orkes

systems outside Conductor, which Orkes leaves you to source and operate. Akka can run alongside existing Conductor workflows while you build the agentic layer on one platform.

Orkes added an agent runtime, guardrails, and an MCP gateway. Doesn't that make it an agentic platform?

Those are real additions. In Orkes an agent is a durable workflow that calls an LLM, with guardrails and approvals as workflow steps and tools exposed through the gateway. Long-term memory, streaming, and the application runtime around it are still yours to assemble. Akka is built for agentic AI end to end — native agents, built-in memory, embedded governance, and a full-stack runtime.

offers no pre-deployment classification. Akka Verify applies any control to any AI system, guarantees its enforcement, and covers pre-deployment governance.

Orkes raised $60M and is built on proven Netflix technology. Isn't that lower risk?

Conductor's orchestration maturity is real and an advantage in that layer. But strong funding and a proven orchestration core do not add the missing agentic layers, and Orkes is scaling on investor capital toward profitability. Akka funds its roadmap from profit, owns one SLA across the whole system, and backs it with indemnities.

Sources

Orkes Series B: $60M Series B, April 23, 2026, led by AXA Venture Partners (businesswire.com/news/home/20260423550324; finsmes.com; thesaasnews.com); prior $20M Series A (2024), $9.3M seed (2022)

Netflix lineage: orkes.io/what-is-conductor; techcrunch.com/2023/12/13/orkes-forks-conductor — Conductor open-sourced at Netflix 2016; Orkes founded 2021 by Conductor creators; Netflix dropped support Dec 2023, Orkes maintains the fork

Orkes Cloud SLA: orkes.io/pricing; orkes.io/cloud — up to 99.99% advertised for Enterprise; orkes.io/cloud-service-level-agreement — Plan A 99.9%, Plan B 99.0%, service credits as sole remedy

Orkes AI / agents: orkes.io/content/developer-guides/ai-orchestration; orkes.io/agentspan; orkes.io/blog/agents-on-conductor-architecture-for-production-ai; docs.conductor-oss.org/devguide/ai/agent-guardrails.html; orkes.io/changelog; orkes.io/content/developer-guides/mcp-api-gateway — Conductor agent runtime and AGENT task (Aug–Sep 2026; Agentspan now part of Conductor), agent and PII guardrails (Aug 2026), Agent Evals, MCP/API Gateway, multi-LLM, vector DB integration, HITL approvals

Orkes persistence / streaming: docs.conductor-oss.org/devguide/running/deploy.html; github.com/conductor-oss/conductor (32.4k stars) — PostgreSQL recommended for production, Redis + Elasticsearch/OpenSearch for throughput, MySQL/Cassandra supported; CDC events to Kafka/SQS/Pub-Sub/NATS; worker-polling task queues

Orkes security: orkes.io/blog/orkes-is-now-soc-2-type-2-compliant; orkes.io/security; orkes.io/changelog — SOC 2 Type II; no HIPAA statement on orkes.io/security or docs.orkes.io/security (checked 2026-10-26); RBAC, configuration audit logs (Aug 2026), secrets

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 II + public SOC 3; annual pen tests, SBOMs, 40+ policies

Akka performance: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster; ~10T vs ~2T tokens/core; ~80% less compute than Python

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 241 regulations and frameworks / 1,718 controls / 699 with financial penalty; 100,000+ deployments / since 2009; profitable; Dell Technologies Capital